Strategic Engineering Blueprint for Winning the Flip the Switch Hackathon

Executive Summary and Event Mechanics

The Flip the Switch Hackathon, co-hosted by WeMakeDevs and DevTools Academy and powered by Switch AI (sandbox-quantum/switch), challenges software builders to transform isolated terminal AI agents into collaborative teammates operating inside existing group chat platforms[1]. With registrations approaching 5,000 participants and historical WeMakeDevs events drawing over 8,500 registrants and 1,200 project submissions, standing out requires far more than a simple question-answering bot[1]. Securing first place requires a precise alignment with the internal architecture of the Switch platform, the evaluation priorities of the judging panel, and a disciplined execution plan that eliminates demonstration failures[2].

The competition operates within a four-day building window running from November 4, 2026 (12:01 AM IST) to November 7, 2026 (11:59 PM IST), preceded by an orientation kickoff livestream on November 2, 2026 (7:30 AM PST / 3:30 PM UK time)[6]. Entry is free and open globally to individuals aged 18 or older, competing either solo or in teams of up to four members[7]. A critical structural advantage of the competition rules is multi-track stacking: a single participant or team can simultaneously submit a primary workflow project, open up to five pull requests in the open-source core contribution track, dictate their entire build by voice for the Wispr Flow side track, publish a technical tutorial for the blog prize, and post build updates on social media[1].

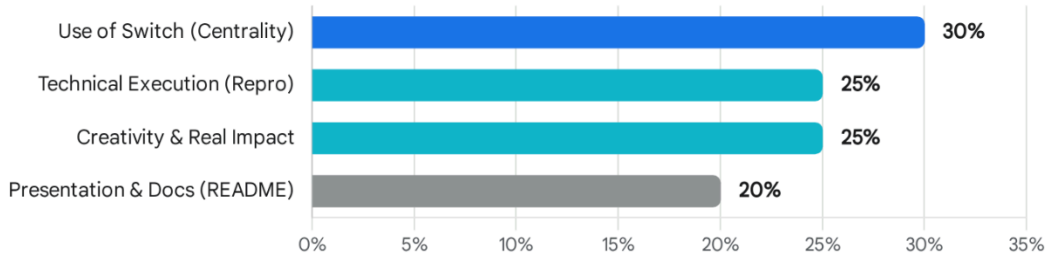
Strategic Allocation of **Judging Weights** and Multi-Track Prize Pools

TOTAL PRIZE POOL	BUILD WINDOW	TRACK 03 LIMIT	TRACK STACKING
$10,000+	**96 Hours**	**5 Open PRs**	**100% Eligible**
Cash & hardware tiers	Nov 4 (12:01 AM) – Nov 7 IST	Max assigned simultaneously	Workflow + PRs + Wispr + Blog

OFFICIAL JUDGING CRITERIA BREAKDOWN

Evaluated by joint panel: Switch, DevTools Academy, and WeMakeDevs

100% Scale

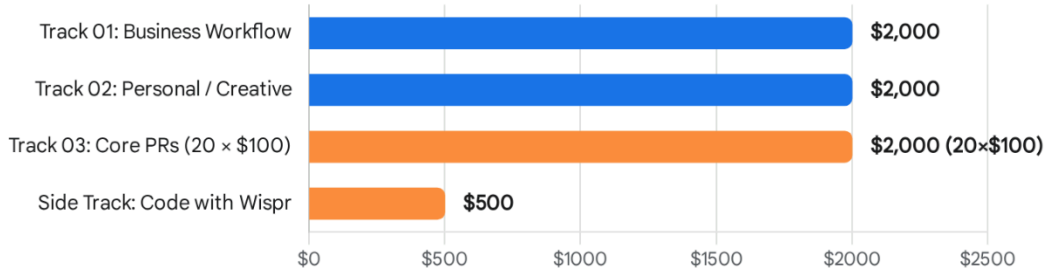


Primary Gate: Switch centrality (30%) + Reproducible deployment (25%) constitute 55% of score.

CASH ALLOCATIONS ACROSS TRACKS

Direct cash rewards stackable across workflow and contribution tracks

$6,500 Direct Cash



STACKABLE HARDWARE & RECOGNITION TIERS

🎧 **AirPods 5** Best Blog Post	⌨️ **Logitech Keyboards** Top 5 #HowISwitch Posts	⭐ **Keychron Keyboard** GitHub Repo Star Raffle	🎁 **Swag + Certificates** 20 Random Packs + All Submissions

The joint judging panel from Switch, DevTools Academy, and WeMakeDevs weights **Switch centrality (30%)** and **reproducible technical execution (25%)** as the primary filters, while multi-track stacking allows a single builder to compete for up to $3,000 in cash plus hardware.

Data sources: WeMakeDevs Hackathon Portal, Event Schedule, Tracks & Guidelines, DevTools Academy Evaluation Rubric

Judging occurs in two distinct phases: WeMakeDevs first reviews all submissions to compile a qualified shortlist, and a joint panel from Switch AI, DevTools Academy, and WeMakeDevs subsequently evaluates the finalists[2]. Official guidelines and engineering advisories published by WeMakeDevs establish that the primary elimination filter is the **Remove-Switch Test**[5]. If a submission consists of a single AI agent answering questions in a chat channel, evaluators classify it as a standard webhook chatbot that does not require Switch to function, resulting in immediate disqualification during shortlisting[5]. To pass this filter and capture the highest possible score, an architecture must coordinate multiple specialized agents running across separate directories or machines alongside human decision-makers, producing a concrete, verifiable end state such as an opened pull request, a verified bug reproduction script, or an approved deployment rollback[5].

Track / Reward Category	Prize Allocation	Core Requirement & Deliverable	Strategic Stackability
Track 01: Best Business Workflow	$2,000 Cash (1st Place)	Practical multi-agent and human workflow solving a real team operational bottleneck[1].	Primary Workflow Target[1]
Track 02: Best Personal / Creative	$2,000 Cash (1st Place)	Everyday life automation, personal project management, or creative production[1].	Alternative Primary Target[1]
Track 03: Best Core Contribution	$2,000 Cash (100 × 20 PRs)	Meaningful code or connector PRs on GitHub issues labelled hackathon by Nov 7, 11:59 PM IST[1].	Fully Stackable (Up to 5 Open PRs)[1]
Side Track: Code with Wispr	$500 Cash	Dictate agent prompts, room rules, and README via Wispr Flow; submit Insights	Fully Stackable with Any Track[1]

		scorecard screenshot[1].	
Side Track: Best Blog	AirPods 5	Publish a comprehensive tutorial walking first-time users through the build and Switch setup[1].	Fully Stackable with Any Track[1]
Social: #HowISwitch	5 × Logitech Mechanical Keyboards	Share build progress and demo clips on social media using #HowISwitch[1].	Fully Stackable[1]
Community & Random Draws	Keychron Keyboard + 20 × Swag	Star the sandbox-quantum/switch repo (Keychron) and submit a valid project (Swag + Certificate)[1].	Automatic Upon Submission[1]

Deep-Dive Switch Architecture and the Gemini Pro Power Stack

Designing a winning system requires a thorough grasp of how Switch operates under the hood so the project highlights the platform's unique strengths. Switch is neither an AI model provider nor a replacement messaging application; it is a self-hosted, open-source infrastructure layer that connects external team chat platforms with local or server-hosted AI agents[1].

Internal Mechanics of the Switch Stack

The Switch platform coordinates human and machine collaboration across four tightly integrated technical layers[12]. At the center sits **Switch Core** (core/switch_core/), a Python FastAPI server backed by PostgreSQL (managed via Alembic migrations and tested with pytest-asyncio) and an internal Matrix message bus (Tuwunel via matrix-nio)[7]. Every Switch room maps persistently to one external chat channel via a PostgreSQL database row so channel bindings survive server restarts, and agent events are stored directly in PostgreSQL rather than single-process memory to prevent state loss[12]. Connecting Switch Core to human workspaces is the **Collaboration Bridge** layer, which ships with five native adapters: Mattermost WebSocket (bundled automatically inside the local Docker Compose stack so builders need no corporate admin access), Slack Socket Mode, Discord Gateway WebSocket,

Telegram Bot API polling, and a Microsoft Teams HTTP listener[1].
On the developer's machine, **Switch Console** (console/) operates as an Electron desktop application maintaining its own local SQLite database to manage agent lifecycles[12]. When a chat message addresses a local coding agent, the Console's watcher claims the room for that session, writes prompts into the agent's local pseudo-terminal (PTY) or remote terminal multiplexer, and serves Model Context Protocol (MCP) tools over HTTP so the agent can interact with the room[16]. Finally, communication between custom agents and the server is governed by the **Switch Agent Protocol** and a centralized authorization module (core/switch_core/authz.py)[12]. Agents maintain one persistent Server-Sent Events (SSE) stream downward to receive room events in real time and issue HTTP POST requests upward for actions across four reachability models (always_on, session_addressable, auto_session, and session_passive)[12]. Because the legacy task delegation protocol was removed in version 0.31+, all inter-agent coordination now occurs through post_message and send_targeted_message[14]. Every agent action passes through authz.py (can(principal, action, resource)), which resolves an agent's request to its human owner's identity and enforces that owner's exact read, write, and delete permissions across Rooms, Documents, References, and Packages[12].

Turning "Google Gemini Pro Only" into a Competitive Advantage

While casual participants may assume that paid subscriptions to Anthropic's Claude Code or OpenAI's Codex are necessary, building exclusively with **Google Gemini Pro** provides a major architectural advantage when deployed across three complementary engines supported by Switch[1].
The first engine is **OpenCode CLI backed by Google Gemini Pro** for local terminal coding sessions. Switch Console natively supports three coding CLIs out of the box: Claude Code, Codex, and OpenCode[1]. OpenCode is an open-source, provider-agnostic terminal agent that authenticates directly with Google Gemini Pro (opencode auth login) using a standard Gemini API key[7]. Switch Console automatically injects its switch-connector-opencode plugin into OpenCode, enabling Gemini Pro to inspect local repositories, run test suites in isolated working directories, modify source code, and report results back to the chat room[14].
The second engine is a **Custom Google ADK and Google GenAI Protocol Agent**. The hackathon rules explicitly highlight custom agents built with **Google ADK** (google.adk / @google/adk) and google-genai joining rooms via the Switch Agent Protocol[1]. By running a lightweight Python service that listens to Switch's SSE stream and invokes google.genai.Client with Gemini Pro, a builder can operate an always_on triage agent alongside on-demand OpenCode coding agents, demonstrating mastery of both out-of-the-box Console providers and custom protocol integrations[12].
The third engine is **SandboxAQ Switch Trust Integration** (switch-trust-cli and switch-trust-sdk-py). Inspection of the sandbox-quantum GitHub organization reveals that the sponsor's commercial security suite—Switch Trust—natively supports Google Gemini (google.genai) and Google ADK (ADKGuardrailsPlugin)[7]. Wrapping the Gemini Pro client with Switch Trust guardrails and invoking switch-trust scan (which maps whitebox code vulnerabilities to the OWASP Top 10 for Agentic Applications with CVSS severity scores)

directly inside the room workflow aligns the project with the sponsor's core enterprise product suite[7].

Critical Repository Bug-Avoidance Protocols

An examination of open issues in sandbox-quantum/switch reveals several active platform bugs that could disrupt a live demonstration if left unaddressed, all of which can be neutralized through proactive architectural choices[5].

GitHub Issue / Platform Constraint	Observed Failure Mechanism in Switch	Mandatory Architectural Workaround
Issue #1158: Mattermost Kickoff Activity Card Bug	On bundled Mattermost, a template's automated kickoff: turn activity card can fail with "Invalid RootId parameter", causing infinite publisher retries[22].	Keep the kickoff: field in template.yaml to a simple plain-text greeting, and initiate complex multi-agent threads via a top-level room message[19].
Issue #1097: Invisible Room Documents in Bridged Chat	Internal room documents created via Switch's document store are not visible when viewing the conversation from a bridged chat client like Mattermost or Slack[22].	Instruct agents via Room Instructions to post all tracebacks, diffs, and scan tables directly as Markdown messages in the chat thread and save files to Git[5].
Issues #1288 & #1289: Console Env Var Stripping	Saved provider environment variables in Switch Console's Advanced Settings can disappear or fail to attach when launching local agent sessions[5].	Persist GEMINI_API_KEY and GOOGLE_GENAI_API_KEY inside OpenCode's native auth store (opencode auth login) and load .env files explicitly in Python scripts[5].
Issue #1226: Long Code Block Splitting	Lines exceeding 2,000 characters inside fenced code blocks are split improperly on bridges like Discord, truncating agent replies[22].	Enforce a room instruction requiring agents to cap terminal traceback blocks at 30 lines (under 1,500 characters) per message[5].
Immutable Agent	An agent's **Name**, **Run**	Finalize the exact folder

Registration Properties	**Location**, and **Working Directory** are permanently fixed at creation; changing them requires deleting and re-registering the agent[5].	tree (/workspace/telemetry, /workspace/sandbox-app, /workspace/release-ops) in template.yaml before registering agents[5].

Track Impact Analysis and Strategic Selection

Selecting the optimal primary track determines how judges evaluate the technical depth, practical impact, and reproducibility of the submission[2].

Strategic Dimension	Track 01: Best Business Workflow ($2,000)	Track 02: Best Personal / Creative ($2,000)	Track 03: Best Core Contribution ($100 × 20)
Core Alignment with Switch Thesis	**Maximum**: Switch is explicitly designed for multi-human, multi-agent team rooms in enterprise chat[11].	**Moderate**: Personal workflows typically involve a single human, making team collaboration harder to demonstrate[1].	**High**: Directly expands Switch's connector catalog (catalog/), messaging adapters, and core stability[1].
Pass Rate on "Remove-Switch Test"	**High**: Naturally justifies multiple agents with distinct permissions (anyone vs owner-agents) and human sign-off gates[5].	**Low to Moderate**: Entries frequently degenerate into single-agent assistants that could run in a standalone terminal[5].	**Not Applicable**: Evaluated strictly on code quality, test coverage (just test), and usefulness of the top PRs[1].
Judge Reproducibility	**Deterministic**: Can bundle a seeded microservice repository and a template.yaml for a 90-second clean-room run[5].	**Fragile**: Personal APIs (Spotify, personal Gmail, WhatsApp) fail when judges clone the repo without personal OAuth keys[2].	**Strict CI Gate**: Verified via just check, just typecheck, just test, and a mandatory demo video in the PR[7].
Recommended	**Primary Flagship**	Avoid as primary;	**Secondary Cash**

Role in Strategy	**Submission**[1]	incorporate creative polish into presentation	**Multiplier** (Submit 2–3 surgical PRs alongside Track 01)[1]

A comparative evaluation confirms that **Track 01 (Best Business Workflow)** is the highest-impact target for the flagship submission[1]. Switch AI's entire product ecosystem—spanning Switch Rooms, Switch Trust, and Switch ROI—is built around organizational workflows such as incident response, cross-timezone engineering handoffs, security guardrails, and release verification[7]. Moreover, Track 02 (Best Personal or Creative Workflow) carries a severe reproducibility penalty: projects that depend on personal email, calendar, or smart-home credentials cannot be executed cleanly when a judge clones the repository on an isolated test machine, directly jeopardizing the 25% Technical Execution weight[2]. Anchoring the primary build in Track 01 with a self-contained, deterministic repository while stacking Track 03 pull requests and the Wispr and Blog side tracks maximizes both win probability and total prize return[1].

Architectural Dry Runs: Evaluating Five Candidate Blueprints

To identify the single strongest architecture capable of defeating thousands of competing submissions, five distinct multi-agent system designs were subjected to dry-run evaluations against the four official judging criteria: Use of Switch (30%), Technical Execution and Reproducibility (25%), Creativity and Impact (25%), and Presentation and Documentation (20%)[2].

Candidate Architecture	Use of Switch (Max 30)	Technical Execution (Max 25)	Creativity & Impact (Max 25)	Presentation & Docs (Max 20)	Composite Score (Max 100)	Dry-Run Verdict
Dry Run 1: Multi-Modal Support & Doc RAG Bot	8 / 30	20 / 25	10 / 25	15 / 20	**53 / 100**	**Rejected** (Fails Remove-Switch Test)[5]
Dry Run 2:	18 / 30	15 / 25	20 / 25	17 / 20	**70 / 100**	**Rejected** (Low

Personal Life & Creative Content Studio						reproducibility; weak agent separation)[2]
Dry Run 4: Multi-Agent FinOps Vendor Reviewer	24 / 30	17 / 25	21 / 25	16 / 20	**78 / 100**	**Rejected** (Text-only output; lacks code execution proof)[5]
Dry Run 3: Automated Dependency Upgrade Desk	26 / 30	23 / 25	14 / 25	18 / 20	**81 / 100**	**Rejected** (Copies the official blog example verbatim)[2]
Dry Run 5: SwitchGuard SRE War Room	**30 / 30**	**24 / 25**	**23 / 25**	**19 / 20**	**96 / 100**	**Selected Winner** (Full multi-directory, multi-engine, gated pipeline)[5]

Analysis of Rejected Candidates (Dry Runs 1–4)

The first dry run evaluated a **Multi-Modal Support and Documentation RAG Bot** where users upload PDFs or ask questions in Mattermost and a Gemini Pro agent retrieves answers from ChromaDB[5]. Although straightforward to implement, this architecture scores only 53/100 because it fails the Remove-Switch Test outright (8/30 on Use of Switch)[5]. As the official WeMakeDevs engineering guide notes, embedding a question and returning vector chunks is a single-agent lookup that could be served by a simple webhook and produces no collaborative task artifact[5].

The second dry run examined a **Personal Life Organizer and Creative Content Studio** for Track 02, pairing an email/calendar agent with a social media drafting agent[1]. This candidate scored 70/100 due to two structural weaknesses: first, both agents operate on unstructured text without needing separate working directories or machines, violating the Switch design rule that two agents should be merged into one unless they require distinct environments or permissions[5]; second, external OAuth dependencies on personal Gmail and Calendar accounts prevent judges from running a clean-room test upon cloning the repo (15/25 on Technical Execution)[2].

The third dry run tested an **Automated Dependency Upgrade Desk**, where a watch agent monitors package releases and a bump agent cuts a branch, runs tests, and waits for human approval before opening a pull request[5]. While technically sound (26/30 on Use of Switch and 23/25 on Technical Execution), this exact workflow was published as the step-by-step tutorial example in the official WeMakeDevs hackathon blog post[5]. Because dozens of teams will submit near-identical clones of this tutorial, it suffers a heavy penalty on Creativity and Impact (14/25), capping its total score at 81/100[2].

The fourth dry run evaluated a **Multi-Agent FinOps and Vendor Contract Reviewer**, where Security, Legal, and Finance agents review a vendor contract in a room and flag budget overlaps[25]. While this aligns with one of Switch's documented business use cases, the workflow relies exclusively on LLM text summarization rather than deterministic code execution or tool verification, scoring 78/100 because it lacks the verifiable compiler or test-suite proof that distinguishes top hackathon winners[5].

Analysis of the Winning Candidate (Dry Run 5: SwitchGuard SRE)

Drawing directly from the patterns of past WeMakeDevs grand prize winners—specifically *LeakLedger* (which won by featuring a deterministic 90-second replay lab and "fail-closed" evidence boundaries)[32], *SRE Sentinel* (which won the WeMakeDevs Future Stack GenAI Hackathon with dynamic MCP tool execution)[33], and *DisputePoint* (which enforced explicit human sign-off when incident evidence conflicted)[34]—**SwitchGuard SRE** achieves a composite score of 96/100[5]. It combines Switch's official triage-pair.template.yaml pattern with the incident rollback scenario featured on the Switch Hackathon homepage and integrates SandboxAQ's Switch Trust security scanner into a single, deterministic war room[5].

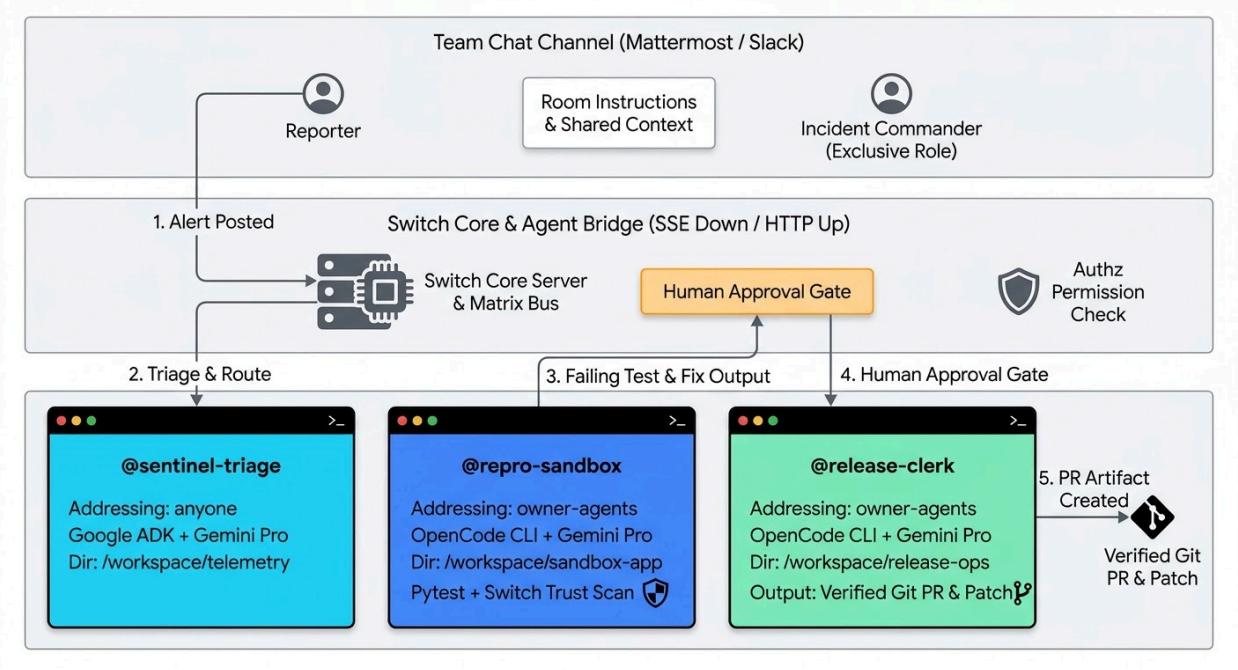
The Winning Project Blueprint: SwitchGuard SRE

Core Concept and the Concrete End Artifact

When a production failure or customer bug report arrives in an engineering chat channel, resolving it normally fragments context across private terminals: one engineer reads the alert in Slack or Mattermost, another tries to reproduce the error locally, a third patches the code, and someone must verify that the hotfix does not introduce new security vulnerabilities before opening a pull request[5].

SwitchGuard SRE unifies this entire incident lifecycle inside a single Switch room where human engineers and three specialized Google Gemini Pro agents collaborate with shared context[11]. Every run begins with an error report or telemetry alert posted in the channel and finishes with

a **definitive, verifiable artifact**: a local Git branch containing a deterministic pytest reproduction script, a verified code fix that passes the full test suite, an OWASP security audit scorecard via Switch Trust, and a packaged Git patch and pull request generated *only* after an authorized human Incident Commander grants explicit approval in the room[5].



Structural Separation of the Three Gemini Pro Agents

To satisfy Switch's architectural requirement that every agent must have a concrete reason to exist separately, SwitchGuard SRE partitions responsibilities across three distinct working directories, two execution harnesses (Google ADK protocol daemon and OpenCode CLI), and two addressing permission tiers (anyone versus owner-agents)[5].

Agent Handle & Room Alias	Engine & Connection Model	Isolated Working Directory	Addressing Policy	Exact Operational Responsibility
{team}-sentinel (Alias: @triage)	Custom Python Agent (google-genai / Google ADK + Switch Trust Guardrails) over Switch Agent Protocol (always_on SSE/HTTP)[12]	/workspace/telemetry (Read-only access to incident logs and ledger; zero access to application source code)[5]	addressing: anyone (Any team member or alert webhook can report an issue to it)[5]	Sanitizes incoming reports for PII, classifies reports into **Bug**, **Question**, or **Incomplete Request**, maintains a live

				markdown table of open incidents, and routes confirmed bugs to @repro via send_targeted_message[14].
{team}-repro (Alias: @repro)	**OpenCode CLI** backed by Google Gemini Pro via Switch Console (auto_session)[1][2]	/workspace/sandbox-app (Isolated clone of the target microservice repository with pytest and switch-trust-cli)[5]	addressing: owner-agents (Protected from random user triggers; responds only to the owner or @triage)[5]	Writes a minimal failing reproduction test, executes pytest, posts the verbatim traceback, writes the minimal code fix, re-runs pytest to verify the fix, runs switch-trust scan, and requests human sign-off[5].
{team}-clerk (Alias: @release)	**OpenCode CLI** backed by Google Gemini Pro via Switch Console (auto_session)[1][2]	/workspace/release-ops (Release repository with Git branch/PR credentials and CHANGELOG.md)[5]	addressing: owner-agents (Strictly gated by the Room Instructions approval rule)[5]	Remains idle until the human holding the exclusive: true Incident Commander role replies Approved. Then commits the verified patch on a hotfix branch, updates CHANGELOG.

				md, and outputs the PR diff link[5].

This three-agent division enforces least-privilege security and operational clarity[5]. Because {team}-sentinel uses addressing: anyone, any employee in the company chat can report a problem to it, but because it lives in /workspace/telemetry, a malicious or accidental prompt cannot alter source code or trigger heavy test builds[5]. Conversely, {team}-repro and {team}-clerk have write access to code and release branches, so they are locked down with addressing: owner-agents, ensuring they only execute work routed by the owner's triage agent or approved directly by the human owner[5].

Encoding Team Conventions in Room Instructions

Rather than duplicating behavioral rules across individual agent system prompts, SwitchGuard SRE places shared operational laws inside the **Switch Room Instructions**[5]. Every agent invited to a Switch room automatically reads the room's instructions upon joining, ensuring that any future agent added via !invite-agent immediately inherits the team's safety boundaries[5].

The Room Instructions establish five core operational rules[5]. First, before executing any slow step such as running pytest, switch-trust scan, or Git operations, an agent must post a one-line announcement stating the exact command it is about to run so human teammates in the channel know the agent is actively working[19]. Second, agents are strictly forbidden from summarizing test runs into vague assurances; they must paste the exact command executed and the verbatim terminal traceback inside a Markdown code block capped at 30 lines (under 1,500 characters) to avoid message-splitting bugs[5]. Third, the room enforces a **Fail-Closed Evidence Boundary**: if @repro cannot deterministically reproduce a reported bug with a failing test in /workspace/sandbox-app, it must stop and report REPRODUCTION_INCONCLUSIVE rather than guessing a fix[19]. Fourth, no pull request or release branch may be created without explicit human approval from the user holding the exclusive: true Incident Commander role (configured with auto_mode: true and auto_approve: false), and if a human rejects a proposed patch, they state the reason in one line so the room preserves the engineering decision[5]. Fifth, all diffs, scan scorecards, and release artifacts must be posted directly in the chat thread and committed to /workspace/release-ops/artifacts/ so bridged chat users can inspect them immediately[5].

The Deterministic 90-Second Demo Scenario ("The Cart-Service Failure")

To guarantee that any judge cloning the repository can reproduce the entire workflow in 90 seconds with zero external cloud setup, /workspace/sandbox-app comes pre-seeded with a lightweight Python FastAPI microservice named cart-service—a deliberate nod to the cart-service 500 error dialogue featured on the official Flip the Switch Hackathon landing page[5].

The repository includes a deterministic replay script (./scripts/trigger_incident.py) and three copy-paste scenarios that demonstrate the full breadth of the architecture[5]. In the primary

scenario, a team member posts in the Mattermost room that cart-service checkout has been returning HTTP 500 errors since deploy v2.14.0 whenever customers apply promotional discount code SAVE20 on multi-item carts, tagging @triage[8]. Immediately, {team}-sentinel (@triage) restates the issue in one line, classifies it as a confirmed bug Report (INC-001), updates the room's live incident status table, and addresses @repro via send_targeted_message with the exact payload parameters to test[14].
Next, {team}-repro (@repro) announces in the thread that it is creating tests/test_cart_discount_bug.py inside /workspace/sandbox-app and running pytest[19]. Seconds later, @repro posts the verbatim failing pytest output showing an unhandled ZeroDivisionError and floating-point precision fault in cart_service/pricing.py[5]. @repro then applies a minimal patch to pricing.py, re-runs pytest (displaying 5 passed in 0.14s), and executes a security verification check using switch-trust scan (alongside a deterministic AST security check fallback), confirming zero OWASP vulnerabilities introduced by the patch[5]. @repro posts the unified git diff in the channel and halts at the approval gate, asking the Incident Commander for sign-off[5].
To demonstrate Switch 0.31+ mid-turn conversation tracking (follow_the_conversation enabled via !follow-ups on), the human engineer first replies in the thread without an @-mention: *"Use Decimal with ROUND_HALF_UP instead of round() so we stay compliant with tax rounding rules."*[14] @repro automatically picks up the untagged human follow-up, updates pricing.py to use Python's decimal module, re-runs pytest, and posts the updated diff[5]. The human Incident Commander then posts: *"Approved. @release cut hotfix branch v2.14.1 and generate the PR artifact."*[5] Finally, {team}-clerk (@release) wakes up in /workspace/release-ops, creates branch hotfix/v2.14.1-inc-001, commits the verified patch, appends the incident summary and rollback instructions to CHANGELOG.md, and posts the final PR artifact in the room[5].

Reproducible Switch Template Specification (switchguard-sre.template.yaml)

The primary technical artifact of the repository is switchguard-sre.template.yaml, modeled directly on Switch's official examples/agent-templates/triage-pair.template.yaml so judges can spin up the entire multi-agent topology in one step[5].

Template Section / Key	Configured Value in switchguard-sre.template.yaml	Architectural Purpose
version	1 (compatible with version: 2 resources: syntax)[14]	Passes Switch Console and Server template schema validation checks[5].
params.team	type: string, pattern: "[a-z][a-z0-9-]*", default:	Dynamically prefixes all created agent names

	sre[19]	(sre-sentinel, sre-repro, sre-clerk) and the room (sre-war-room)[19].
params.provider	type: provider, default: [opencode, claude, codex][19]	Prioritizes opencode (authenticated with Google Gemini Pro) so it is selected automatically on launch[15].
params.bridge	type: bridge, default: [$first][19]	Automatically mirrors the Switch room to bundled Mattermost or connected Slack/Discord[1].
agents[0] ({team}-sentinel)	addressing: anyone, directory: "{$agents_dir}/telemetry"[19]	Public intake and triage agent that classifies reports and routes bugs to {team}-repro[5].
agents[1] ({team}-repro)	addressing: owner-agents, directory: "{$agents_dir}/sandbox-app"[19]	Isolated bug reproduction, pytest runner, and Switch Trust security scanner[5].
agents[2] ({team}-clerk)	addressing: owner-agents, directory: "{$agents_dir}/release-ops"[19]	Gated Git patch and release note publisher that executes only after human sign-off[5].
room	name: "{team}-war-room", users: ["{$creator}"], agents: [...][19]	Places all three agents and the human deployer into a single shared room with a clean plain-text kickoff: prompt[19].

Multi-Track Prize Stacking Strategy (Track 03, Wispr, and Best Blog)

Beyond competing for the $2,000 Track 01 prize, executing a coordinated multi-track strategy allows the same engineering effort to compete for an additional $2,500 in cash plus AirPods 5 and mechanical keyboards[1].

Track 03: Best Core Contribution ($100 × 20 PRs = $2,000 Pool)

Track 03 rewards the top 20 pull requests solving issues labelled hackathon in sandbox-quantum/switch with $100 each[1]. Maintainer @petr-sandbox clarified in issue #829 that issues are no longer exclusively locked to a single assignee; instead, issues are assigned to everyone who comments with a plan, and the best pull request per issue is selected after submissions close on November 7 at 11:59 PM IST[5]. Participants may maintain up to five open pull requests at a time[1].

Inspection of active pull requests and issues in sandbox-quantum/switch (such as #829 Stripe, #819 Notion, #825 PagerDuty, #828 Cloudflare, and #1172 LaunchDarkly) demonstrates that the highest-leverage contributions follow the **Service Connections v2 Catalog Pattern** (core/switch_core/connections/catalog/<vendor>/)[22]. Adding a v2 connector typically requires zero custom broker code—only a declarative connection.yaml file (specifying OAuth dynamic registration or API key authentication, strict HTTPS, PKCE S256, and read-only default scopes), a skill/SKILL.md file (defining safe operational guidelines and ensuring credentials never leak into transcripts or logs), an official SVG logo credited in NOTICE.md, and loader validation tests in core/tests/switch_core/connections/test_loader.py[27]. Builders who prefer core Python or TypeScript bug fixes can alternatively target reproducible platform issues such as #1158 (Mattermost kickoff activity card RootId error), #1226 (Discord code block splitting over 2,000 characters), or #1285 (!list-documents admin client package store fix)[22]. Every Track 03 pull request must be preceded by a two-line plan comment tagging @petr-sandbox, pass just check (ruff), just typecheck (mypy), and just test (pytest against PostgreSQL), include a short screen-recording video demonstrating the fix or connector in action, and have the Contributor License Agreement signed via the exact PR comment: I have read the CLA Document and I hereby sign the CLA[1].

Side Track: Code with Wispr ($500 Cash) and Best Blog (AirPods 5)

To capture the $500 **Code with Wispr** prize, the builder should activate the free one-month Wispr Flow Pro trial (promo_code=WEMAKEDEVS) before writing any code[1]. By using Wispr Flow voice dictation to speak all prompts to OpenCode and Gemini Pro, dictate the room instructions inside switchguard-sre.template.yaml, author the README.md, and speak live incident commands in Mattermost during testing, the builder accumulates a high voice-dictation word count and WPM metric[1]. Exporting a screenshot of the **Insights scorecard** from the Wispr Flow desktop app and embedding it in both the project README and the submission form fulfills the side-track requirement[1].

Simultaneously, the **Best Blog** track (awarding AirPods 5) is evaluated on clarity, accuracy, and usefulness to a first-time Switch user[1]. Publishing a step-by-step technical walkthrough titled *"Building an Autonomous Multi-Agent SRE War Room in Switch with Google Gemini Pro and OpenCode"* that documents the 10-minute Docker/Mattermost setup, OpenCode Gemini authentication, directory isolation, and template capture turns the project documentation into a winning blog entry[1]. Sharing short demo videos of the multi-agent chat handoffs on X and LinkedIn with the hashtag #HowISwitch also enters the project into the draw for five Logitech mechanical keyboards[1].

Day-by-Day Execution Plan and Presentation Blueprint

Front-loading environment setup and Track 03 issue claims during the pre-hackathon window (November 2–3) ensures that the official four-day hacking window (November 4–7) is dedicated entirely to multi-agent orchestration, clean-room testing, and presentation polish[5].

Date & Phase	Core Engineering Milestones	Verification & Submission Deliverable
Pre-Hackathon (Nov 2 – Nov 3)	Register on WeMakeDevs, star sandbox-quantum/switch, join Discord, install Docker, Switch Console, OpenCode CLI, and Wispr Flow; authenticate Gemini Pro (opencode auth login); deploy the official Switch expert room; comment on 2–3 hackathon GitHub issues tagging @petr-sandbox[1].	Local Mattermost active; @switch-expert answering in room; Wispr Flow tracking voice dictation; Track 03 issues assigned[1].
Day 1: Wednesday, Nov 4	Create the fixed agent directory structure (/workspace/telemetry, /workspace/sandbox-app, /workspace/release-ops); seed /workspace/sandbox-app with the cart-service bug and pytest suite; register sre-sentinel, sre-repro, and sre-clerk; submit Track 03 PR #1 after passing just check, just typecheck, and just test[5].	Seeded cart-service bug reproduces locally; first agent responds in Mattermost; Track 03 PR #1 open with CLA signed[5].
Day 2: Thursday, Nov 5	Write the Switch Room Instructions (encoding the traceback rule, fail-closed rule, and approval gate);	Full triage-to-reproduction loop running in Mattermost; verbatim pytest tracebacks and security scan tables

	rehearse the handoff between @triage and @repro; integrate google-genai and switch-trust scan into the workflow; configure aliases (@triage, @repro, @release) and enable !follow-ups on[5].	rendering cleanly[5].
Day 3: Friday, Nov 6	Connect @release (sre-clerk) behind the Incident Commander approval gate (auto_mode: true, auto_approve: false); capture and validate switchguard-sre.template.yaml via GET /rooms/{id}/yaml; submit Track 03 PR #2[1].	End-to-end flow produces a verified Git branch, patch diff, and CHANGELOG.md entry upon human "Approved" message[5].
Day 4: Saturday, Nov 7 (Closes 11:59 PM IST)	Execute a morning clean-room dry run by wiping the room (just reset), importing switchguard-sre.template.yaml from scratch, and running ./scripts/setup_demo.sh; record the 120-second demo video; embed the Wispr Insights scorecard in the README; publish the blog post and #HowISwitch social posts; submit before 11:59 PM IST[1].	Clean-room run verified; repository, video, blog, Wispr scorecard, and Track 03 PRs submitted ahead of the 11:59 PM IST deadline[5].

The 120-Second Judge Video and README Structure

Because Presentation and Documentation account for 20% of the total score and heavily influence the initial WeMakeDevs shortlisting pass, the two-minute demonstration video and repository README must communicate the value proposition without friction[2].

Video Timestamp	Visual Focus on Screen	Narrative Proof Delivered to Judges
0:00 – 0:15	Split-screen of bundled Mattermost war room and Switch Console showing three Gemini Pro agents online[15].	**Passes Remove-Switch Test Immediately**: States the concrete artifact produced (a verified Git hotfix PR) and shows why three separate directories and permission tiers are required[5].
0:15 – 0:50	Engineer posts the cart-service checkout 500 alert; @triage classifies INC-001 and routes to @repro; @repro runs pytest and switch-trust scan[8].	**Proves Real Code Execution**: Shows verbatim failing pytest traceback, automated patch application, passing test run, and OWASP security verification in the thread[5].
0:50 – 1:30	@repro halts at the approval gate; human dictates mid-turn Decimal refinement and final "Approved" sign-off via Wispr Flow; @release cuts the branch and PR[1].	**Proves Human-in-the-Loop Governance & Wispr Integration**: Demonstrates !follow-ups on mid-turn steering, the mandatory human approval gate, and the final Git PR artifact[5].
1:30 – 2:00	Clean-room import of switchguard-sre.template.yaml inside Switch Console and a quick view of the README and ./scripts/setup_demo.sh[5].	**Guarantees Judge Reproducibility (25% Weight)**: Shows that any evaluator can clone the repo and launch the entire multi-agent room in one click[2].

Conclusions and Strategic Recommendations

Winning the Flip the Switch Hackathon out of thousands of entrants does not depend on building the most complex frontend dashboard or paying for the most expensive proprietary coding models[5]. Instead, victory is determined by how deeply the submission embraces Switch's core thesis: turning isolated agents into a coordinated, permission-gated team inside a

shared chat room[5].
By constructing **SwitchGuard SRE**, the builder directly addresses every criterion on the joint panel's scorecard[2]. Combining **OpenCode CLI** and **Google ADK** powered by **Google Gemini Pro** with **SandboxAQ's Switch Trust** security scanner showcases the full power of the sponsor's open-source and enterprise stack at zero model cost[1]. Enforcing strict directory isolation (/workspace/telemetry, /workspace/sandbox-app, /workspace/release-ops), distinct addressing boundaries (anyone versus owner-agents), a fail-closed reproduction requirement, and a human Incident Commander approval gate guarantees maximum marks on the 30% Use of Switch and 25% Creativity and Impact criteria[2]. Finally, packaging the entire setup into a validated switchguard-sre.template.yaml alongside a deterministic cart-service sandbox—while simultaneously stacking Track 03 catalog pull requests, Wispr Flow voice dictation, and a beginner-friendly technical blog—positions the submission as a standout contender across every prize category[1].

Works cited

1. Flip the Switch Hackathon | WeMakeDevs x DevTools Academy x, https://www.wemakedevs.org/hackathons/switch
2. Flip the Switch Hackathon | DevTools Academy, https://www.devtoolsacademy.com/hackathons/flip-the-switch
3. switch/CODE_OF_CONDUCT.md at main - sandbox-quantum - GitHub, https://github.com/sandbox-quantum/switch/blob/main/CODE_OF_CONDUCT.md
4. The Hangover Part AI: A Week of Building AI That Doesn't Forget, https://www.wemakedevs.org/blogs/hangover-wrap-up-blog
5. How to Build a Switch Workflow That Finishes a Real Task, https://www.wemakedevs.org/blogs/how-to-build-a-switch-workflow-that-finishes-a-real-task
6. Schedule | Flip the Switch Hackathon - WeMakeDevs, https://www.wemakedevs.org/hackathons/switch/schedule
7. https://www.wemakedevs.org/hackathons/switch#tracks
8. Resources | Flip the Switch Hackathon - WeMakeDevs, https://www.wemakedevs.org/hackathons/switch/resources
9. Rules | Flip the Switch Hackathon - WeMakeDevs, https://www.wemakedevs.org/hackathons/switch/rules
10. Flip the Switch Hackathon - Wooble, https://wooble.org/companies/wooble-programs/challenges/meta-global-ai-developer-hackathon
11. GitHub - sandbox-quantum/switch: Open-source AI teammates in, https://github.com/sandbox-quantum/switch
12. switch-agent-human-chat-bridge.md - GitHub, https://github.com/FlorianBruniaux/claude-code-ultimate-guide/blob/main/docs/resource-evaluations/switch-agent-human-chat-bridge.md
13. CONTRIBUTING.md - sandbox-quantum/switch - GitHub, https://github.com/sandbox-quantum/switch/blob/main/CONTRIBUTING.md

14. switch/CHANGELOG.md at main · sandbox-quantum/switch - GitHub, https://github.com/sandbox-quantum/switch/blob/main/CHANGELOG.md
15. Get started with Switch Rooms - Switch AI, https://docs.switchagents.ai/switch-rooms/getting-started
16. Switch Console and the agent runtime — how sessions start, https://docs.switchagents.ai/switch-rooms/internals/switch-console
17. switch/README.md at main · sandbox-quantum/switch · GitHub, https://github.com/sandbox-quantum/switch/blob/main/README.md
18. GitHub - sandbox-quantum/switch-trust-sdk-py: FlintAI Python SDK, https://github.com/sandbox-quantum/switch-trust-sdk-py
19. triage-pair.template.yaml - sandbox-quantum/switch - GitHub, https://github.com/sandbox-quantum/switch/blob/main/examples/agent-templates/triage-pair.template.yaml
20. switch-trust-sdk-ts/tsup.config.ts at main - sandbox-quantum - GitHub, https://github.com/sandbox-quantum/switch-trust-sdk-ts/blob/main/tsup.config.ts
21. Switch Trust CLI · GitHub - sandbox-quantum, https://github.com/sandbox-quantum/switch-trust-cli
22. Issues · sandbox-quantum/switch - GitHub, https://github.com/sandbox-quantum/switch/issues
23. env.example - sandbox-quantum/switch-trust-sdk-ts · GitHub, https://github.com/sandbox-quantum/switch-trust-sdk-ts/blob/main/.env.example
24. Meet Switch - Switch AI, https://docs.switchagents.ai/switch-rooms/using
25. Use cases - Switch AI, https://docs.switchagents.ai/switch-rooms/use-cases
26. Pull requests · sandbox-quantum/switch - GitHub, https://github.com/sandbox-quantum/switch/pulls
27. Add the Stripe connector · Issue #829 · sandbox-quantum/switch, https://github.com/sandbox-quantum/switch/issues/829
28. WeMakeDevs | 4538 members | Space | Connect, https://builder.aws.com/connect/space/29f78273-9a25-34b1-818f-eec9b0bacb31/wemakedevs
29. Gauri Khedekar GauriKhedekar - GitHub, https://github.com/GauriKhedekar
30. GitHub - temburuakhil/WeMakeDevs-Hackathon, https://github.com/temburuakhil/WeMakeDevs-Hackathon
31. The Agent Harness Hackathon - WeMakeDevs, https://www.wemakedevs.org/hackathons/trueforge
32. LeakLedger - Environmental hacks hackathon submission - GitHub, https://github.com/madhavkant29/LeakLedger
33. aryan877/sre-sentinel: devops sre agent, winner of ... - GitHub, https://github.com/aryan877/sre-sentinel
34. DisputePoint Response Intelligence — WeMakeDevs Environmental, https://github.com/vaibhav11123/disputepoint-ship
35. Create a room - Switch AI, https://docs.switchagents.ai/switch-rooms/getting-started/create-a-room
36. WeMakeDevs <> Stream Hackathon: If you're looking to learn new,

https://wemakedevs-stream.devpost.com/